

Computer-Assisted Formalization of Mathematics

Week 01 - An introduction to Lean

First Half

- What is computer-assisted verification (why are we here)
- Class Logistics
- Lean4 and verifiers

Second Half

- Getting acquainted with lean
- First tactics
- The Natural Numbers game

Jaume de Dios Pont - jdedios@nyu.edu

<https://nyulean26.github.io/>

Computer-verified proofs:

Using computers to
verify that a proof is
correct down to the axioms.

Why?

Using computers to
verify that a proof is
correct down to the axioms.

- Why?
 - Correctness

Using computers to
verify that a proof is
correct down to the axioms.

- Why?
 - Correctness
 - We make mistakes
 - AI makes mistakes
 - Software has bugs

Computer assisted verification of proofs

Using computers to
verify that a proof is
correct down to the axioms.

- Why?
 - Correctness
 - We make mistakes
 - AI makes mistakes
 - Software has bugs

- Some proofs are too big
- We rely on deep black boxes
- Most papers have typos
- How do we trust our collaborators?
 - Mathematics at scale

A proof of the Kepler conjecture

Pages 1065-1185 from Volume 162 (2005), Issue 3 by *Thomas C. Hales*

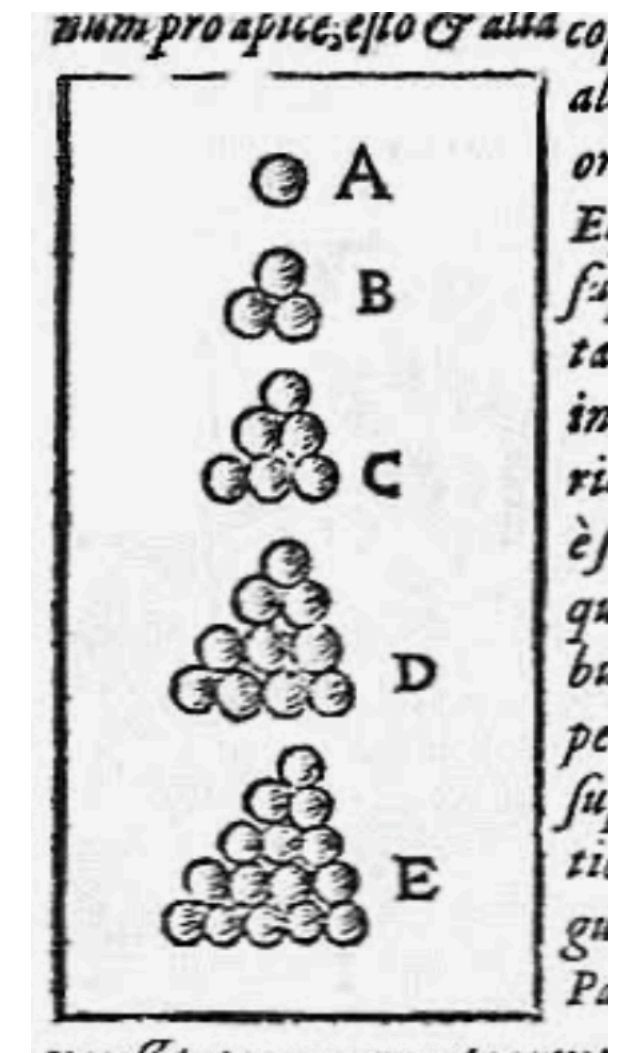
Milestones

Received: 4 September 1998

Revised: 14 February 2003

Accepted: 16 August 2005

Formalized in 2014 by
Project Flyspeck



Computer assisted verification of proofs

Using computers to
verify that a proof is
correct down to the axioms.

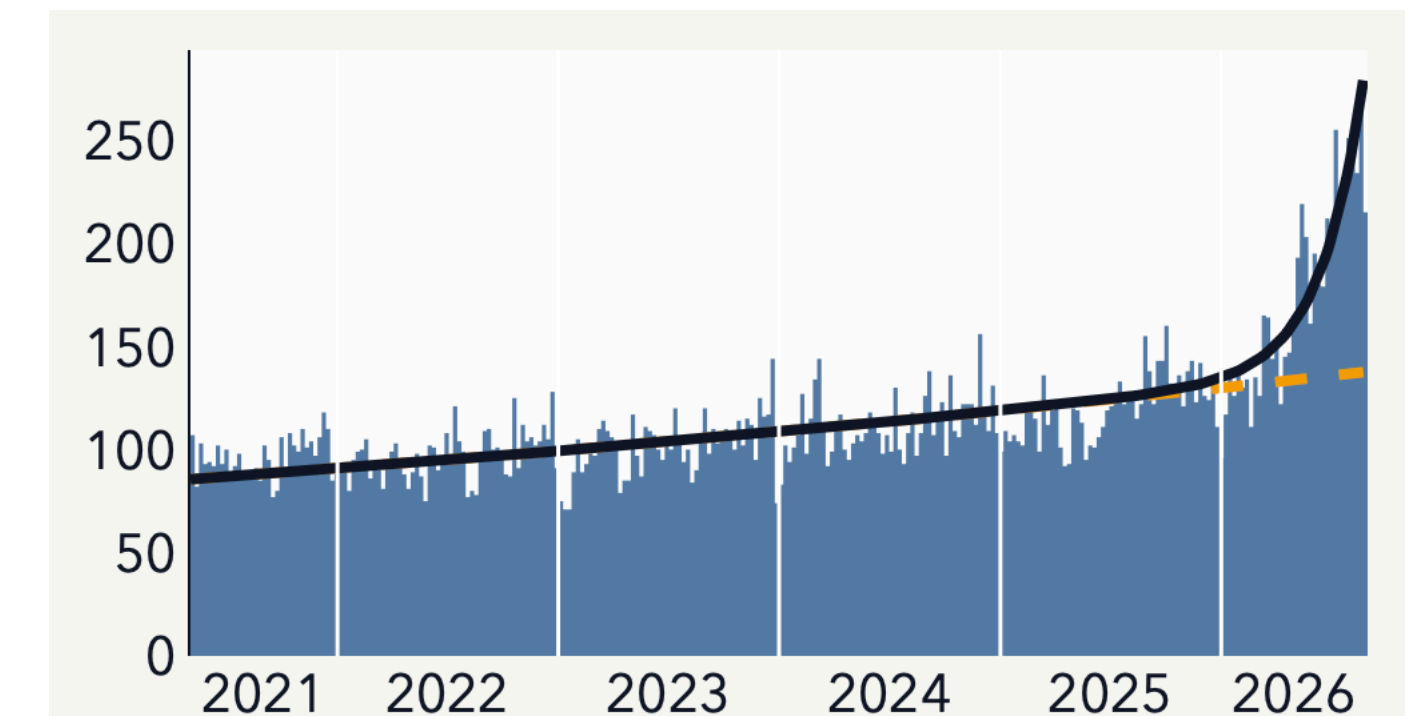
– Why?

– Correctness

– We make mistakes

– AI makes mistakes (At scale!)

– Software has bugs



Weekly submissions to Math.CO

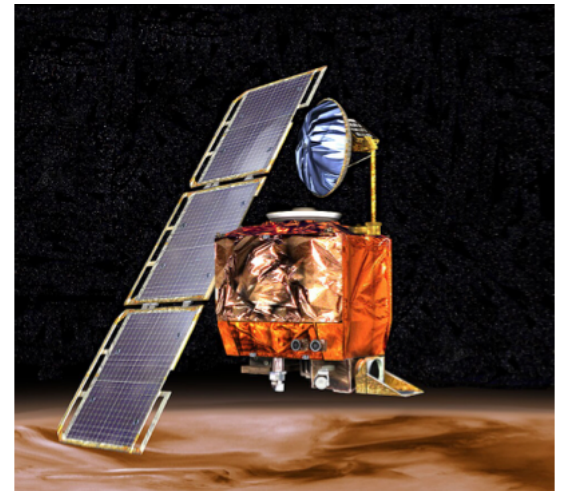
- Mathematics is being produced at an unexpected pace.
- How much are we willing to invest to understand a potentially wrong LLM proof?
- How do we train LLMs beyond human understanding?

Computer assisted verification of proofs

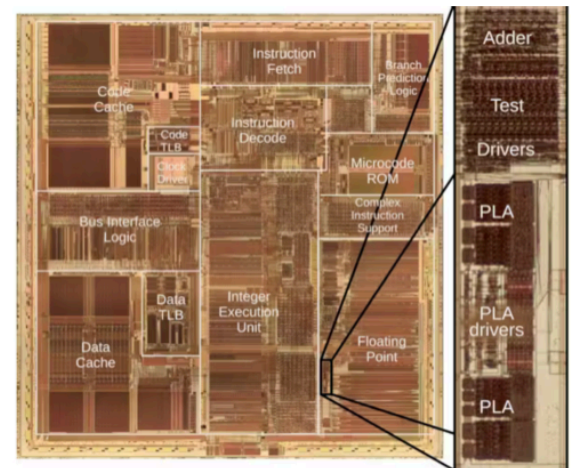
Using computers to
verify that a proof is
correct down to the axioms.

- Why?
 - Correctness
 - We make mistakes
 - AI makes mistakes
 - Software has bugs
- (And they have consequences)

Mars climate orbiter crashed due to a miss-specification in the units provided by a measure system.



The Pentium Fdiv hardware bug cost Intel ~ 1B USD (inflation adjusted)



The Heartbleed software bug affected ~ 1/5th of internet servers, and allowed attackers to obtain passwords and sensitive data.



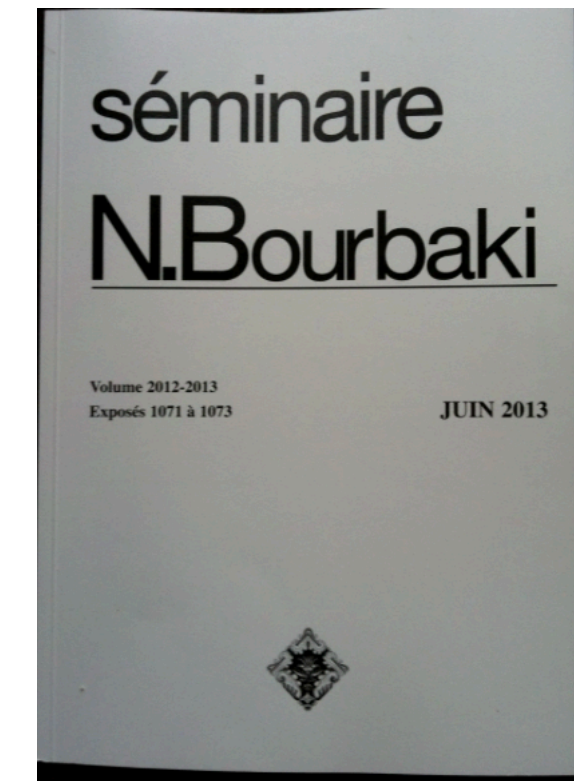
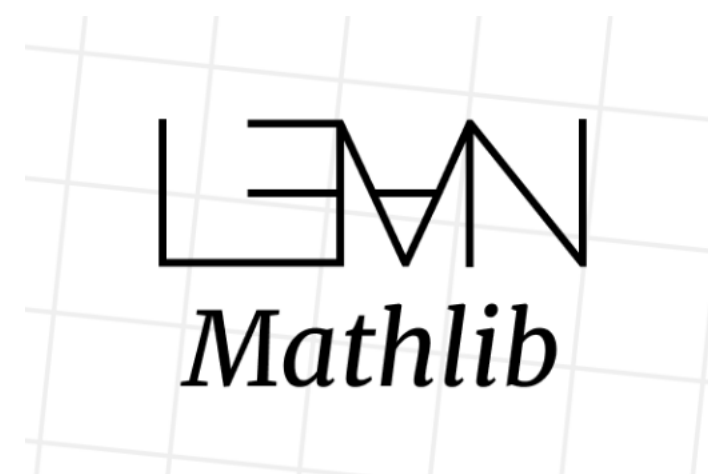
Ethiopian Airlines Flight 302 crash, the deadliest in history, was caused by a bug in the flight stabilisation software.



Computer assisted verification of proofs

Using computers to
verify that a proof is
correct down to the axioms.

- Why?
 - Correctness
 - Understanding

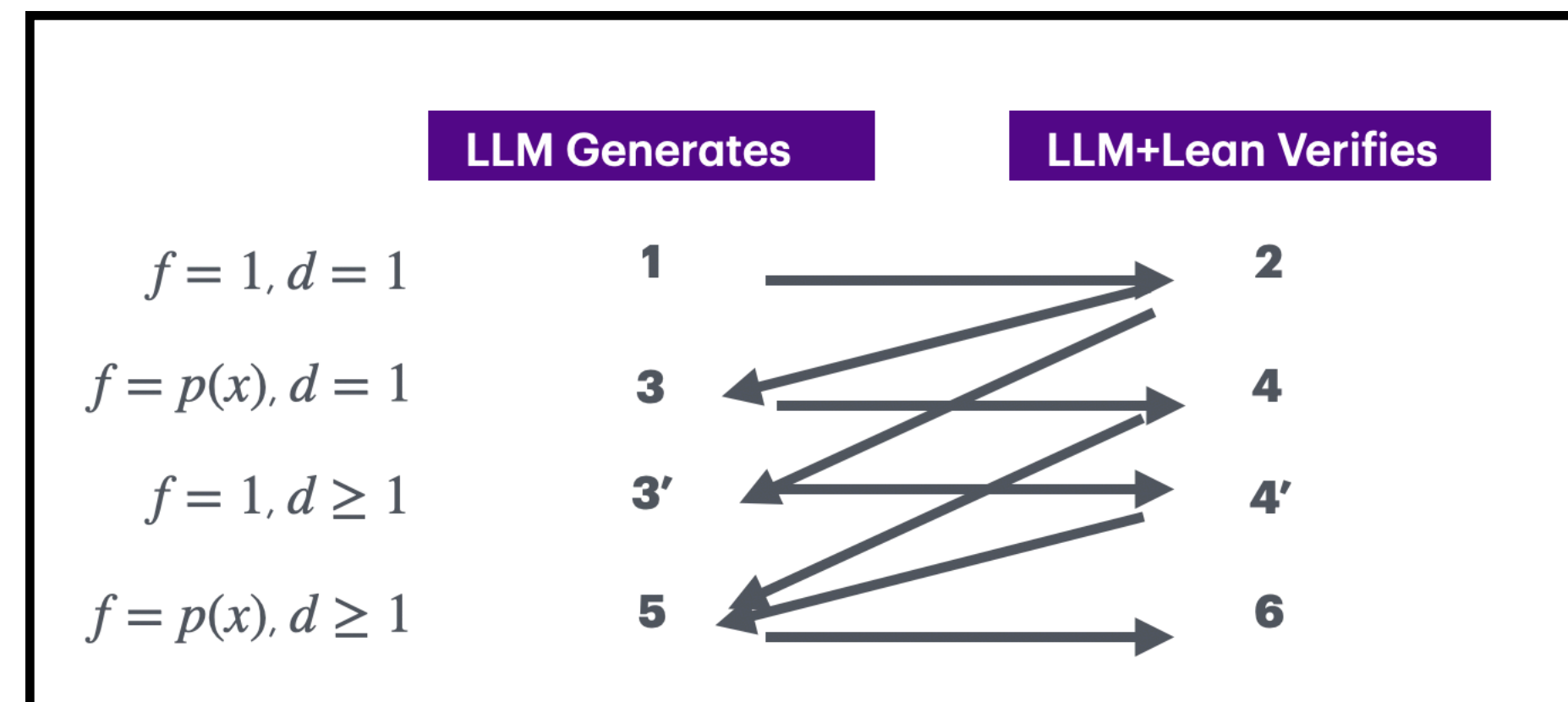


Formal proofs expose *pain points*
of wrong abstractions

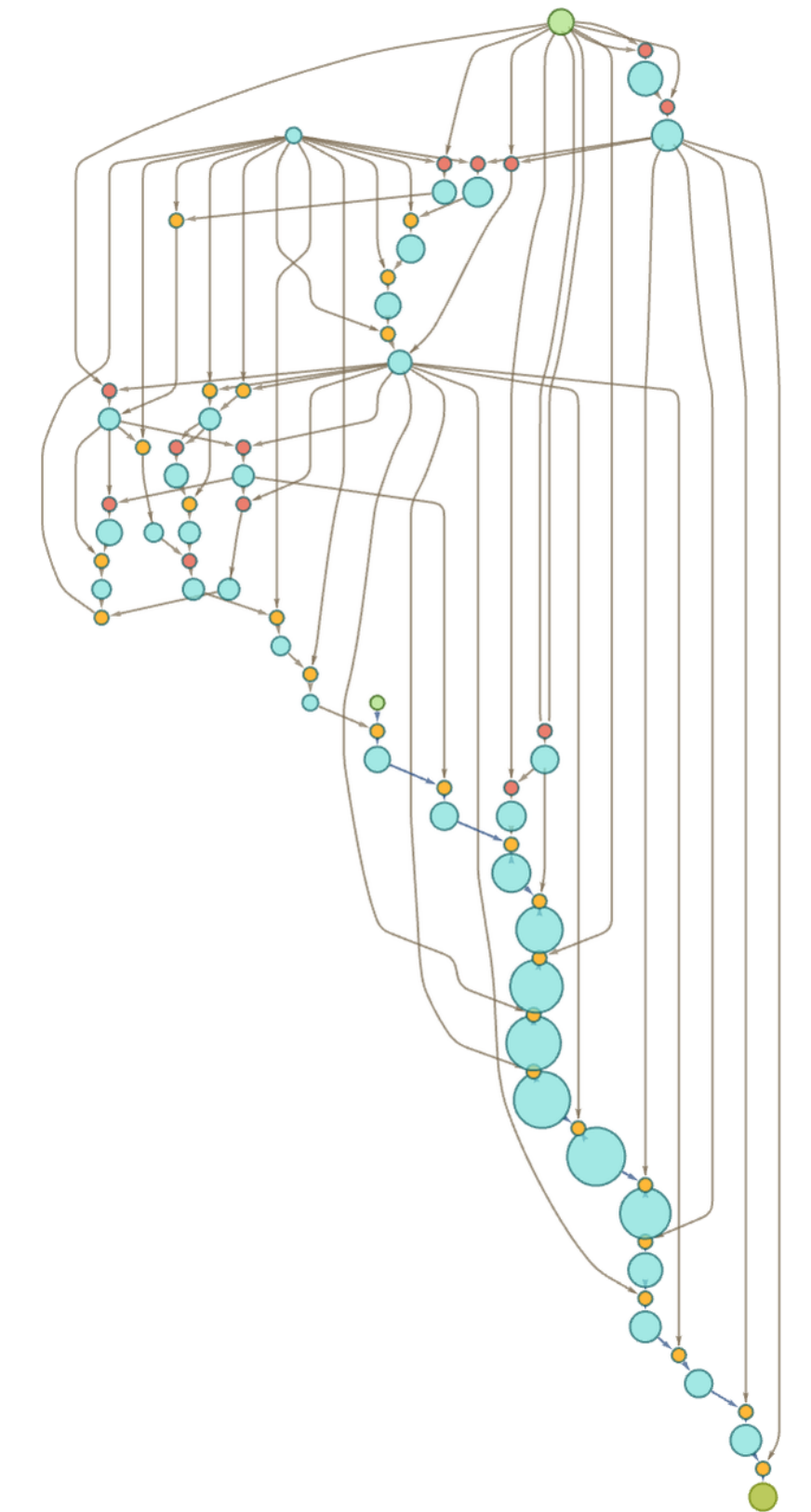
Computer assisted verification of proofs

Using computers to
verify that a proof is
correct down to the axioms.

- Why?
 - Correctness
 - Understanding
 - Automated Search



LEAN as grounding for multi-step arguments that do not fit in LLM (or Human) context

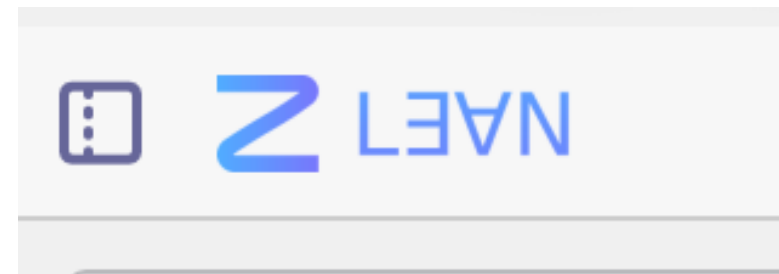


Brute-force searching proofs with 100s of cases.

Computer assisted verification of proofs

Using computers to
verify that a proof is
correct down to the axioms.

- Why?
 - Correctness
 - Understanding
 - Automated Search
 - Because it's fun



Natural Number Game

The classical introduction game for Lean.



In this game you recreate the natural numbers $\mathbb{N}$ from the Peano axioms, learning the basics about theorem proving in Lean.

This is a good first introduction to Lean!

Prerequisites

Computer-verified proofs: How?

(Some logistics for the class)

Our goals in this class (i.e. why are we here?)

- Learn about how these theorem checkers work
- Learn how to write proofs and reusable abstractions
- Learn how to automate proof generation

Our goals in this class (i.e. why are we here?)

- Learn about how these theorem checkers work
- Learn how to write proofs and reusable abstractions
 - What axioms of math do these systems use?
 - How does the “checking” work?
- Learn how to automate proof generation

Our goals in this class (i.e. why are we here?)

- Learn about how these theorem checkers work
 - Learn how to write proofs and reusable abstractions
 - Learn how to automate proof generation
- Learning the proof language, writing proofs by hand.
 - Learning the statement language (with lots of examples!) & common patterns to write reusable statements

Our goals in this class (i.e. why are we here?)

- Learn about how these theorem checkers work
- Learn how to write proofs and reusable abstractions
- Learn how to automate proof generation
- Creating your own tactics (small proof search software)
- Potential pitfalls of automation.
- Using state-of-the-art language models effectively to write Lean proofs.
- How is Lean used in LLM Training?

How will we achieve & evaluate our goals?

- How will the class be organized?
 - Lectures
 - Present a theory concept
 - Live-code an application
 - Lab sessions*
 - Work through examples
 - Learn advanced topics.

* Potentially split in the last 2/3 of the semester.

How will we achieve & evaluate our goals?

- How will the class be organized?

- Lectures

- Present a theory concept
- Live-code an application

- Lab sessions*

- Work through examples
- Learn advanced topics.

* Potentially split in the last 2/3 of the semester.

- How will the class be evaluated?

- 4x in-person quizzes

During the Lab sessions

Weeks 3, 5, 7, 9

Make-ups on week 10

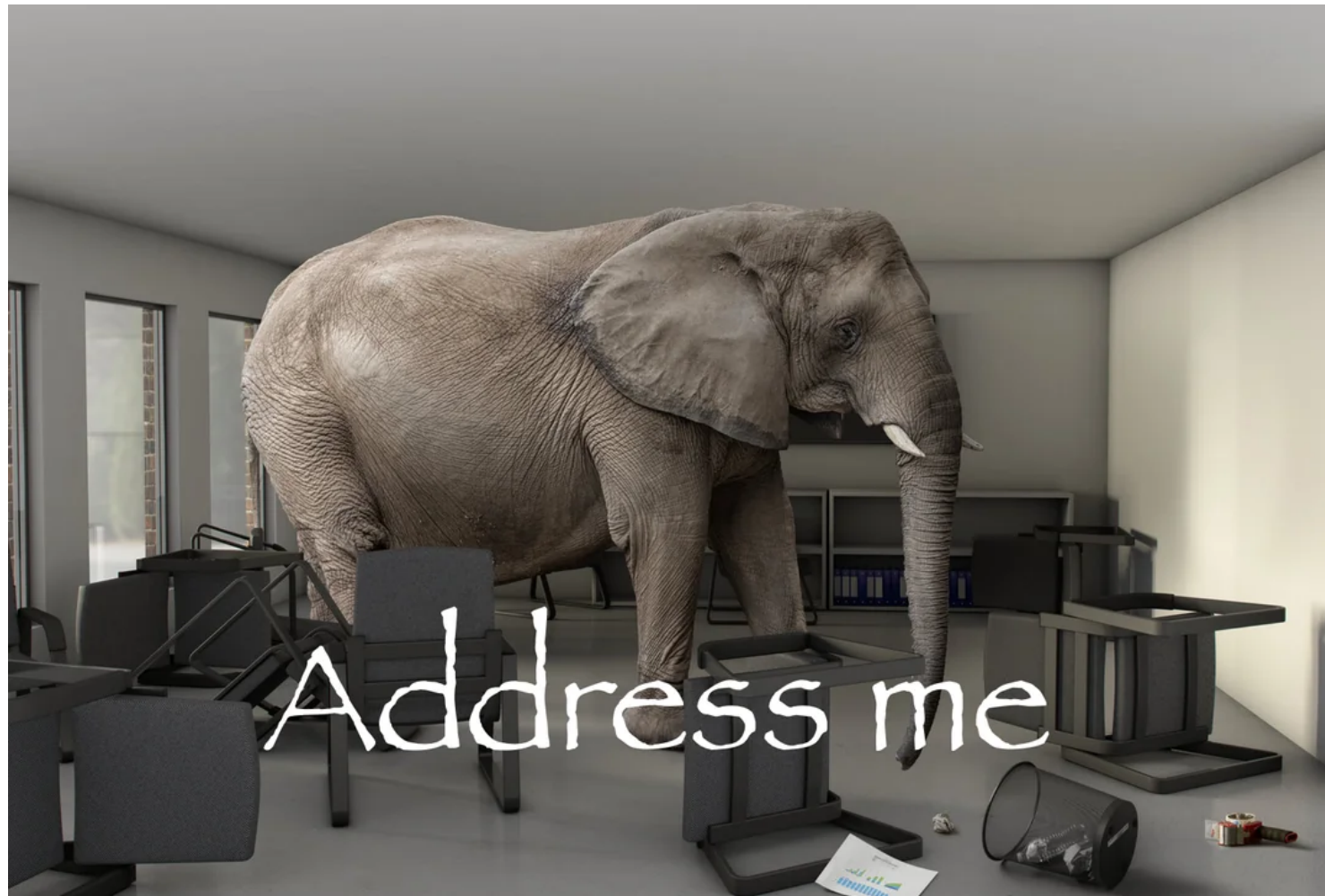
- 4x Homeworks

Due Weeks 3, 5, 7, 9

- One final project

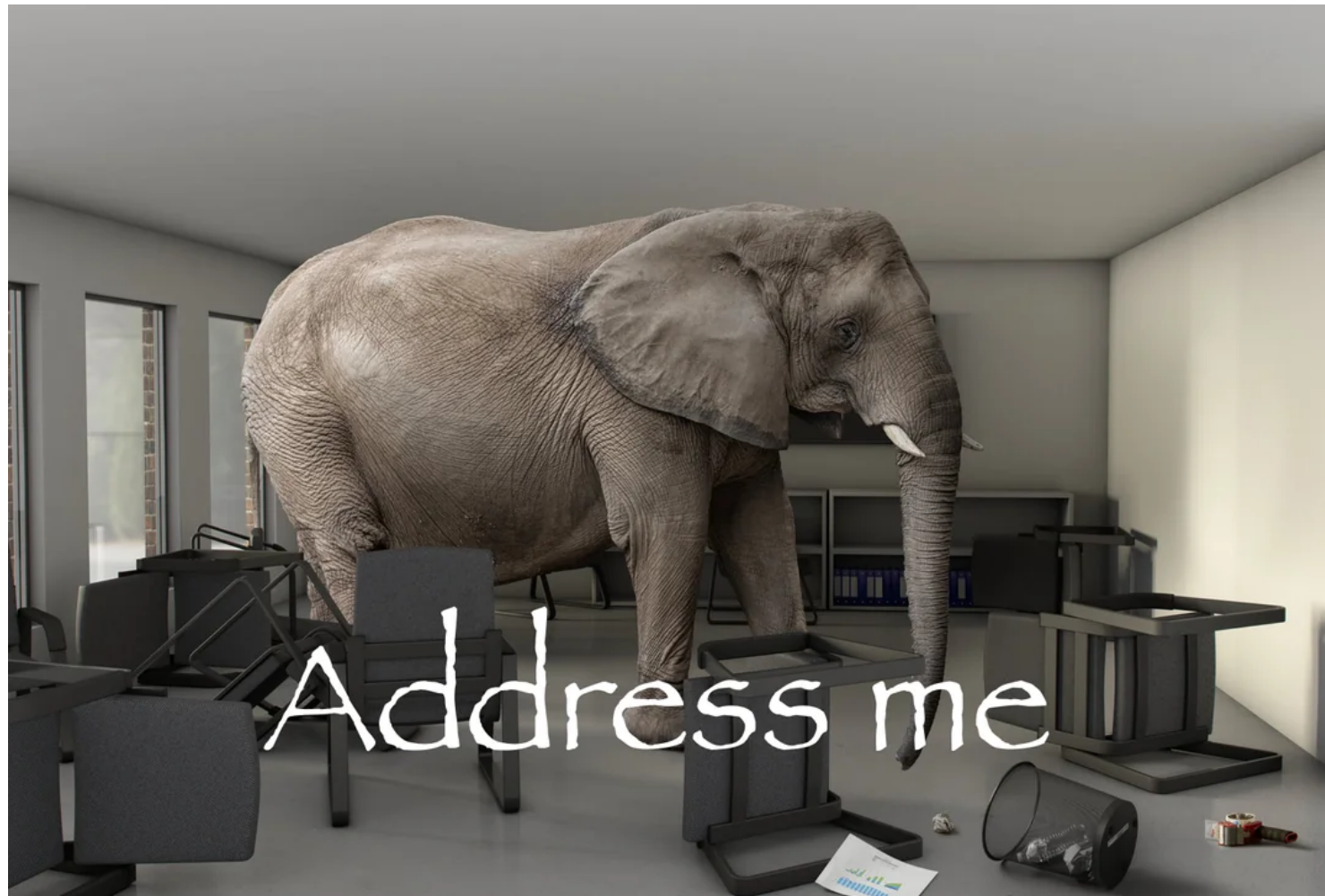
(More info coming Week 5)

AI Usage in this class



Yes. LLMs can write Lean
that compiles efficiently

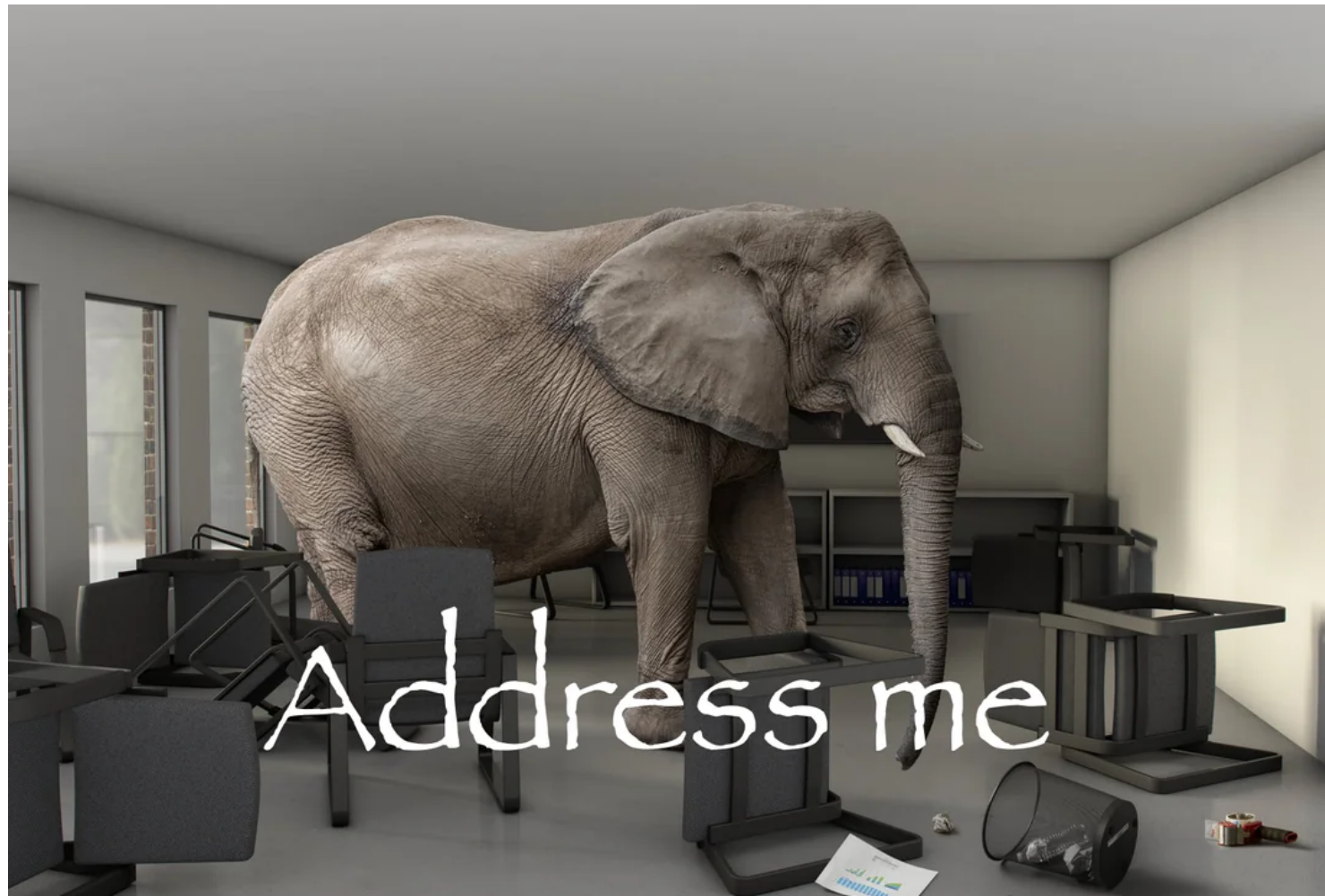
AI Usage in this class



Yes. LLMs can write Lean that compiles efficiently

- The goal of the first weeks is to learn the Language.
(It's about you! => **No LLM**)
- The goal of the final project is to build something cool that you can defend.
(**LLMs yes**, but be ready to answer questions)

AI Usage in this class



Yes. LLMs can write Lean
that compiles efficiently

- The goal of the first weeks is to learn the Language.
(It's about you! => **No LLM**)
- The goal of the final project is to build something cool that you can defend.
(**LLMs yes**, but be ready to answer questions)

You are strongly encouraged to play with different systems to test their capabilities on your own math problems!

Byproducts of formalization & deliberate AI Usage

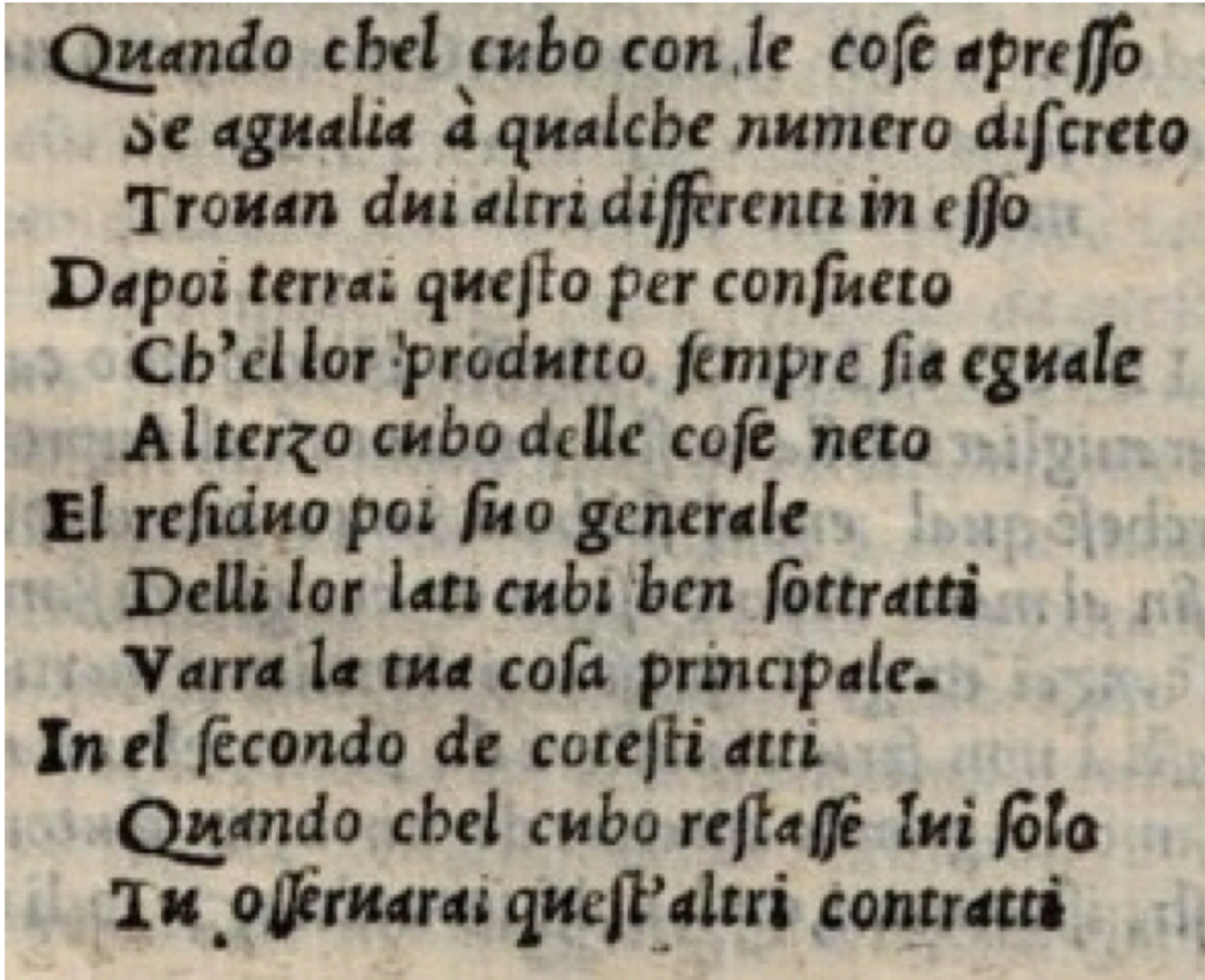
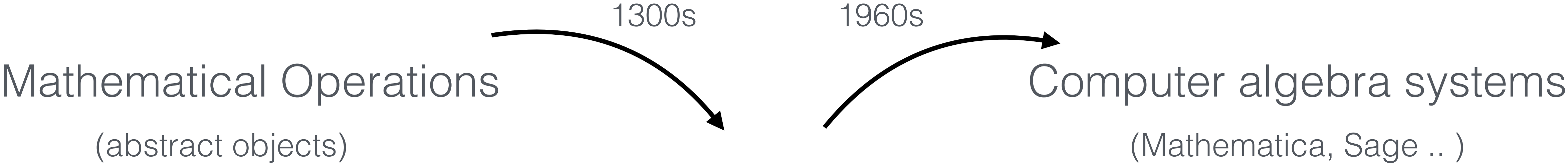
- What is your goal with formalization?
 - I want to know whether a statement is true. => Go LLMs*
 - I want to create a reusable library of definitions and facts. => Careful human-agent joint development
 - I want to understand what is the best way to formalize a theory. => >50% human-edited
Lots of blackboard time

* always check the statements & definitions!!

Interactive Theorem provers

The software we use to
write computer proofs.

Computer algebra systems



Equations
(Symbolic representation)

$$\begin{aligned} \int_{r_1}^{r_2} \left| \frac{d}{dt} u_t \right| dt &\leq \frac{1}{n\alpha(n)} \int_{r_1}^{r_2} \frac{1}{t^{n-1}} \left(\int_{\partial B_t} |\nabla u| d\mathcal{H}^{n-1} \right) dt \\ &= \frac{1}{n\alpha(n)} \left[\frac{1}{t^{n-1}} \int_{B_t \setminus B_{r_1}(0)} |\nabla u| d\mathcal{L}^n \right]_{r_1}^{r_2} + \frac{n-1}{n\alpha(n)} \int_{r_1}^{r_2} \frac{1}{t^n} \left(\int_{B_t \setminus B_{r_1}(0)} |\nabla u| d\mathcal{L}^n \right) dt \\ &= \frac{1}{n\alpha(n)} \frac{1}{r_2^{n-1}} \int_{B_{r_2} \setminus \overline{B}_{r_1}(0)} |\nabla u| d\mathcal{L}^n + \frac{n-1}{n\alpha(n)} \int_{r_1}^{r_2} \int_{B_{r_2} \setminus \overline{B}_{r_1}(0)} \frac{|\nabla u(x)|}{t^n} 1_{\{r_1 \leq |x| \leq t\}} d\mathcal{L}^n(x) dt. \end{aligned} \tag{2.28}$$

```
^ In[4]:= PowerExpand@expr
Out[4]= 
$$\frac{\frac{6 i \sqrt{2} \sqrt{y}}{\sqrt{2 \sqrt{3}-3 x}} - \frac{3 i \sqrt{6} x \sqrt{y}}{\sqrt{2 \sqrt{3}-3 x}} + \sqrt{6} \sqrt{-2 \sqrt{3}+3 x} \sqrt{y}}{12 \sqrt{\pi} x}$$


^ In[3]:= FullSimplify[
  PowerExpand@expr,
  x ∈ Integers && x ≥ 2 && y > 0
]
0
```

What is a formal proof?

- **Statements:** formulas in a proof language.
- **Axioms:** basic formulas we believe.
- **Deduction rules:** ways to transform formulas.

Derive a **Statement** from **Axioms** using **Deduction Rules**

- Proofs must be systematically (mechanically) verifiable.
- Example A and $A \implies B$ can be turned into B

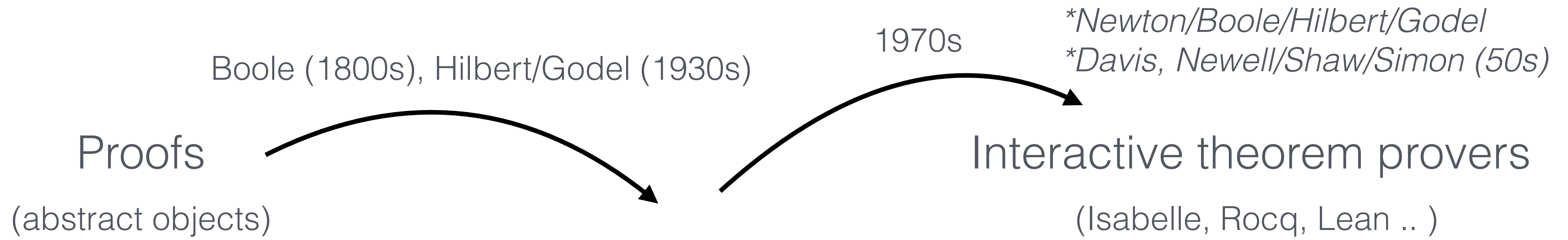
Formal proofs can be manipulated like equations

$$P \wedge (Q \wedge R) \implies (P \wedge Q) \wedge R$$

1.	$P \vee (Q \vee R)$		
2.	$\neg P$		
3.	$P \vee Q$	✓ $\vee$ Intro	2
4.	$(P \vee Q) \vee R$	✓ $\vee$ Intro	3
5.	$\neg Q \vee R$		
6.	$\neg Q$		
7.	$P \vee Q$	✓ $\vee$ Intro	6
8.	$(P \vee Q) \vee R$	✓ $\vee$ Intro	7
9.	$\neg R$		
10.	$(P \vee Q) \vee R$	✓ $\vee$ Intro	9
11.	$(P \vee Q) \vee R$	✓ $\vee$ Elim	5,6-8,9-10
12.	$(P \vee Q) \vee R$	✓ $\vee$ Elim	1,2-4,5-11

From Math.SE user Bram28

Interactive Theorem Provers by analogy



Formal proof objects (Symbolic representation)

THE SQUARE ROOT OF
ANY PRIME IS IRRATIONAL

IF $\sqrt{P}$ IS RATIONAL THEN:

$$\sqrt{P} = \frac{A}{B} \Leftrightarrow P = \frac{A^2}{B^2} \Leftrightarrow B^2 P = A^2$$

REGARDLESS OF THE NUMBER OF PRIME FACTORS
OF A OR B , A^2 AND B^2 WILL HAVE
AN EVEN NUMBER OF PRIME FACTORS.

THE NUMBER OF PRIME FACTORS IN $B^2 P$ IS ODD
BUT THE NUMBER OF PRIME FACTORS OF A^2 IS EVEN
SINCE THE FACTORIZATION INTO PRIMES IS UNIQUE
WE REACHED A CONTRADICTION!

- Ax. 1. $(P(\varphi) \wedge \Box \forall x(\varphi(x) \Rightarrow \psi(x))) \Rightarrow P(\psi)$
- Ax. 2. $P(\neg \varphi) \Leftrightarrow \neg P(\varphi)$
- Th. 1. $P(\varphi) \Rightarrow \Diamond \exists x \varphi(x)$
- Df. 1. $G(x) \Leftrightarrow \forall \varphi (P(\varphi) \Rightarrow \varphi(x))$
- Ax. 3. $P(G)$
- Th. 2. $\Diamond \exists x G(x)$
- Df. 2. $\varphi \text{ ess } x \Leftrightarrow \varphi(x) \wedge \forall \psi (\psi(x) \Rightarrow \Box \forall y (\varphi(y) \Rightarrow \psi(y)))$
- Ax. 4. $P(\varphi) \Rightarrow \Box P(\varphi)$
- Th. 3. $G(x) \Rightarrow G \text{ ess } x$
- Df. 3. $E(x) \Leftrightarrow \forall \varphi (\varphi \text{ ess } x \Rightarrow \Box \exists y \varphi(y))$
- Ax. 5. $P(E)$
- Th. 4. $\Box \exists x G(x)$

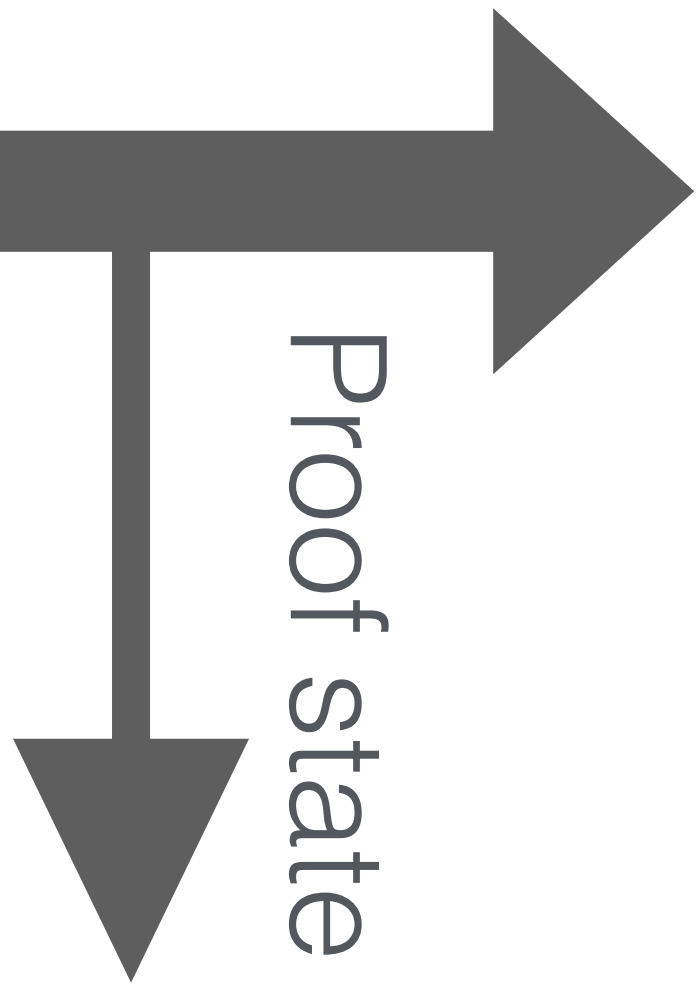
```
theorem squares
  (a b : ℝ) -- variables
  (h0: (a ≥ 0) ∧ (b ≥ 0)) : --hypotheses
  (a+2*b)^2 ≥ a^2+b^2 := by -- goal
    ring_nf -- Simplify in rings
    bound -- hammer for inequalities in ℝ
```


Inside an Interactive Theorem Prover

Lean4 Code

```
theorem squares :
  (a b : ℝ) -- variables
  (h0 : (a ≥ 0) ∧ (b ≥ 0)) : --hypotheses
  (a + 2 * b) ^ 2 ≥ a ^ 2 + b ^ 2 := by -- goal
  ring_nf -- Simplify in rings
  bound -- hammer for inequalities in ℝ
```

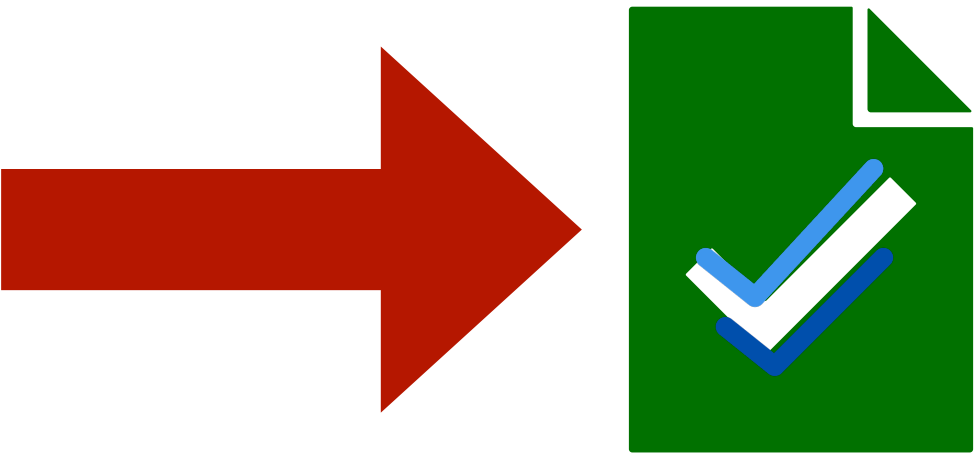
Elaborator



Core Lean4 Expressions

```
theorem squares : ∀ (a b : Real)
(h0 :
  And (@GE.ge.{0} Real Real.instLE a (@OfNat.ofNat.{0} Real (nat_lit 0) (@Zero.toOfNat0.{0} Real Real.instZero)))
  (@GE.ge.{0} Real Real.instLE b (@OfNat.ofNat.{0} Real (nat_lit 0) (@Zero.toOfNat0.{0} Real Real.instZero))),
@GE.ge.{0} Real Real.instLE
  (@HPow.hPow.{0, 0, 0} Real Nat Real (@instHPow.{0, 0} Real Nat (@Monoid.toNatPow.{0} Real Real.instMonoid))
    (@HAdd.hAdd.{0, 0, 0} Real Real Real (@instHAdd.{0} Real Real.instAdd) a
      (@HML.hMul.{0, 0, 0} Real Real Real (@instHML.{0} Real Real.instMul)
        (@OfNat.ofNat.{0} Real (nat_lit 2)
          (@instOfNatAtLeastTwo.{0} Real (nat_lit 2) Real.instNatCast
            (@Nat.instAtLeastTwoHAddOfNat (@OfNat.ofNat.{0} Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
              (@Nat.instNeZeroSucc (@OfNat.ofNat.{0} Nat (nat_lit 0) (instOfNatNat (nat_lit 0))))))))
        b))
    (@OfNat.ofNat.{0} Nat (nat_lit 2) (instOfNatNat (nat_lit 2))))
  (@HAdd.hAdd.{0, 0, 0} Real Real Real (@instHAdd.{0} Real Real.instAdd) a
    (@HPow.hPow.{0, 0, 0} Real Nat Real (@instHPow.{0, 0} Real Nat (@Monoid.toNatPow.{0} Real Real.instMonoid)) a
      (@OfNat.ofNat.{0} Nat (nat_lit 2) (instOfNatNat (nat_lit 2))))
    (@HPow.hPow.{0, 0, 0} Real Nat Real (@instHPow.{0, 0} Real Nat (@Monoid.toNatPow.{0} Real Real.instMonoid)) b
      (@OfNat.ofNat.{0} Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))))) :=
fun (a b : ...) (h0 : ...) =>
@Eq.mpr.{0}
  (@GE.ge.{0} Real Real.instLE
    (@HPow.hPow.{0, 0, 0} Real Nat Real (@instHPow.{0, 0} Real Nat (@Monoid.toNatPow.{0} Real Real.instMonoid))
      (@HAdd.hAdd.{0, 0, 0} Real Real Real (@instHAdd.{0} Real Real.instAdd) a
        (@HML.hMul.{0, 0, 0} Real Real Real (@instHML.{0} Real Real.instMul)
          (@OfNat.ofNat.{0} Real (nat_lit 2)
            (@instOfNatAtLeastTwo.{0} Real (nat_lit 2) Real.instNatCast
              (@Nat.instAtLeastTwoHAddOfNat (@OfNat.ofNat.{0} Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
                (@Nat.instNeZeroSucc (@OfNat.ofNat.{0} Nat (nat_lit 0) (instOfNatNat (nat_lit 0))))))))
          b))
            (@OfNat.ofNat.{0} Nat (nat_lit 2) (instOfNatNat (nat_lit 2))))
      (@HAdd.hAdd.{0, 0, 0} Real Real Real (@instHAdd.{0} Real Real.instAdd) a
        (@HPow.hPow.{0, 0, 0} Real Nat Real (@instHPow.{0, 0} Real Nat (@Monoid.toNatPow.{0} Real Real.instMonoid)) a
          (@OfNat.ofNat.{0} Nat (nat_lit 2) (instOfNatNat (nat_lit 2))))
        (@HPow.hPow.{0, 0, 0} Real Nat Real (@instHPow.{0, 0} Real Nat (@Monoid.toNatPow.{0} Real Real.instMonoid)) b
          (@OfNat.ofNat.{0} Nat (nat_lit 2) (instOfNatNat (nat_lit 2))))))
    (@GE.ge.{0} Real Real.instLE
```

(Small, trusted)
Kernel



▼ Tactic state

1 goal

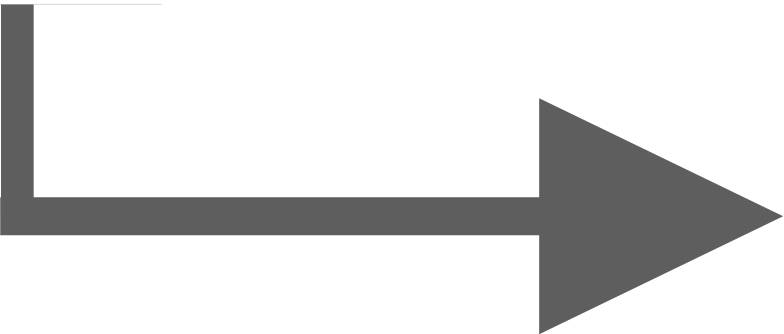
a b : ℝ

h0 : a ≥ 0 ∧ b ≥ 0

└ a * b * 4 + a ^ 2 + b ^ 2 * 4 ≥ a ^ 2 + b ^ 2

Side view

Compiler



Executable
code (C/VM)

The first taste of Lean

You can already read Lean statements

```
theorem exists_root {f :  $\mathbb{C}[X]$ } (hf : 0 < degree f) :  $\exists z : \mathbb{C}, \text{IsRoot } f z := \text{by}$ 
```

R is a type (like a set)

R has a commutative
Ring Structure

x and **y** are elements of type **R**

```
lemma difference_of_cubes {R : Type} [CommRing R] (x y : R) :  
  (x^2 + x*y + y^2) * (x - y) = x^3 - y^3 := by  
  ring
```

: separates hypotheses and conclusion

:= -> states you will give a proof

ring is a *tactic*
(proof automation)

“by” enters *tactic (proof) mode*

```
def Irrational (x :  $\mathbb{R}$ ) :=  
  x  $\notin$  Set.range (( $\uparrow$ ) :  $\mathbb{Q} \rightarrow \mathbb{R}$ )
```

```
theorem norm_inner_le_norm { $\mathbb{k}$  E : Type*} [RCLike  $\mathbb{k}$ ] [SeminormedAddCommGroup E]  
  [InnerProductSpace  $\mathbb{k}$  E] (x y : E) :  
   $\| \langle x, y \rangle \| \leq \|x\| * \|y\| := \text{by}$ 
```

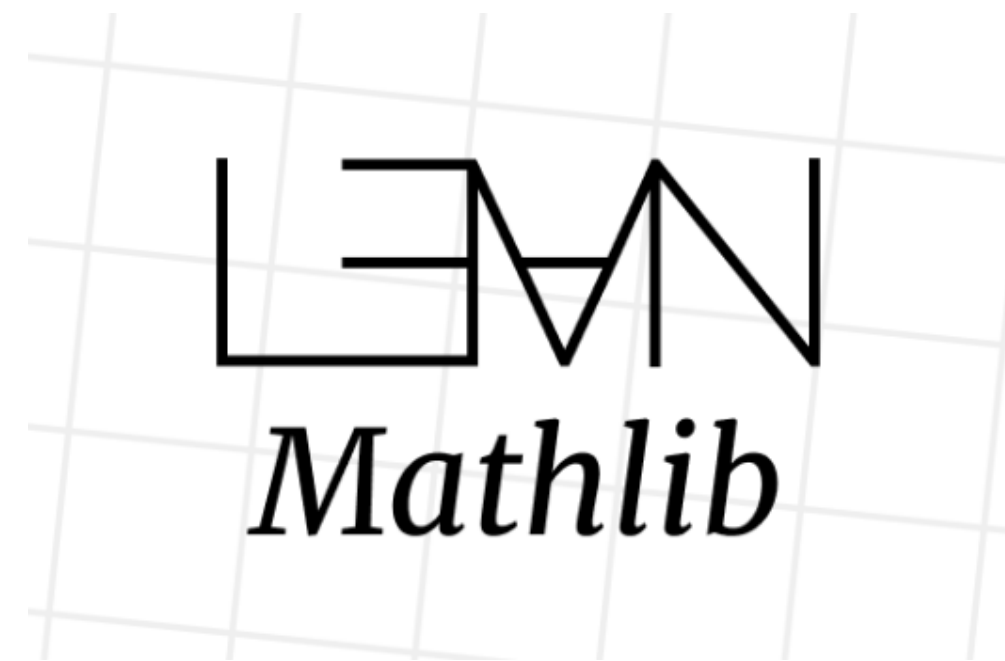
```
theorem irrational_sqrt_two : Irrational ( $\sqrt{2}$ ) := by  
  simp using Nat.prime_two.irrational_sqrt
```


Where do I get my theorems?

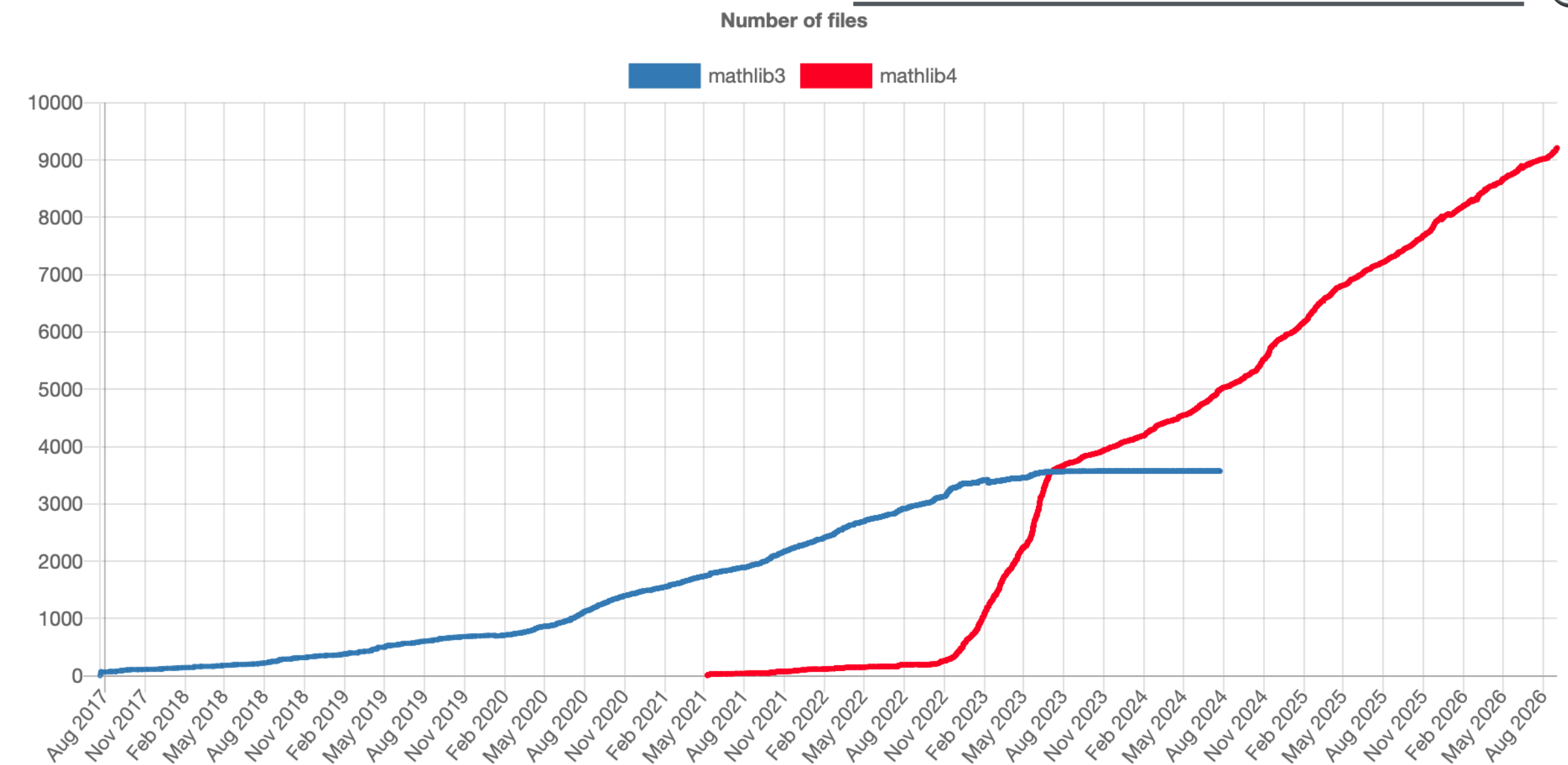
- Mathlib
- TauCeti / LeanPool / Palomar
- Other internet projects

Where do I get my theorems?

- Mathlib
- TauCeti / LeanPool / Palomar
- Other internet projects



Mathlib Overview



- Golden standard
 - Extensively reviewed
- Bourbaki-Style
- Almost all undergrad math

Where do I get my theorems?

- Mathlib
- TauCeti / LeanPool / Palomar
- Other internet projects



- Mostly AI-Generated code
- AI-refereed statements (+Humans sometimes)
- Useful for theorems, sketchy for definitions.

Where do I get my theorems?

- Mathlib
- TauCeti / LeanPool / Palomar
- Other internet projects
 - Extremely high variance
 - Initiatives in specific topics
 - PhysLib, CSLib, TorchLean..

How do I run Lean?

- The VSCode/VSCodium Way

1. Install VSCode

2. Install the VSCode Lean extension

- In the terminal (only for agents)

1. Ask your agent to install Lean

2. Install the lean-lsp-mcp

- The quick and dirty way

1. Go to live.lean-lang.org

- To play the games

1. <https://adam.math.hhu.de/>

- **Today**

Go to NYULean26.github.io and click on the link under week 1

- **This weekend**

Play the Natural Number Game at <https://adam.math.hhu.de/>

Install Lean on your laptop